

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

A REPORT ON

ADVANCED GENERATIVE AI AND ITS APPLICATIONS IN DATA SCIENCE AND SOFTWARE ENGINEERING

submitted by

DIVYANSHU SHARMA (23BDS1155)

in partial fulfilment for the award of

B. Tech. Computer Science and Engineering

(Specialization in Data Science)

School of Computer Science and Engineering

Vellore Institute of Technology, Chennai

October - 2026

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

DECLARATION

I hereby declare that the project entitled "ADVANCED GENERATIVE AI AND ITS APPLICATIONS IN DATA SCIENCE AND SOFTWARE ENGINEERING" submitted by me to the School of Computer Science and Engineering, Vellore Institute of Technology, Chennai Campus, Chennai 600127 in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a record of bonafide work carried out by me. I further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma of this institute or of any other institute or university.

Signature of the Candidate

DIVYANSHU SHARMA (23BDS1155)

Place: Chennai

Date: 13-10-2026

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

CERTIFICATE

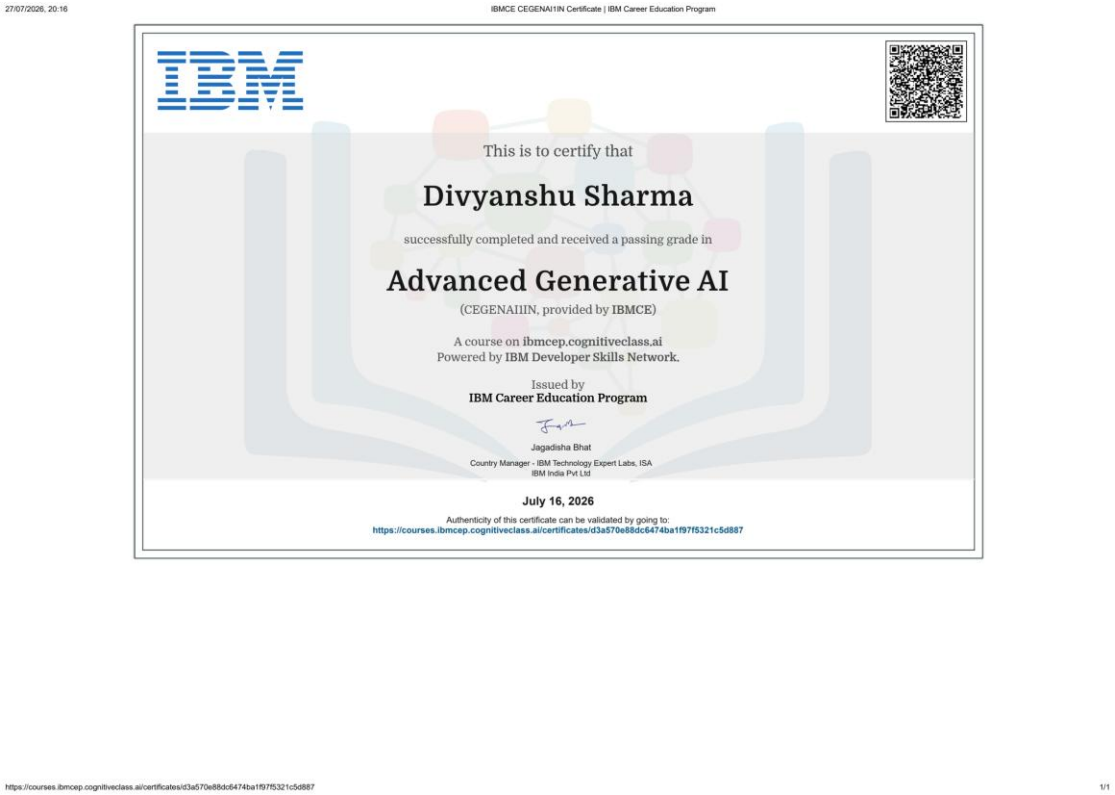
The project report entitled “ADVANCED GENERATIVE AI AND ITS APPLICATIONS IN DATA SCIENCE AND SOFTWARE ENGINEERING” is prepared and submitted by Divyanshu Sharma (Register No: 23BDS1155). It has been found satisfactory in terms of scope, quality and presentation as partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering in Vellore Institute of Technology, Chennai, India.

Examined by:

Examiner I **Examiner II**

INDUSTRY INTERNSHIP CERTIFICATE

This is to certify that Divyanshu Sharma (Register No: 23BDS1155) has completed the technical internship and corporate curriculum entitled “Advanced Generative AI” (Course Code: CEGENAI1IN) conducted by the IBM Career Education Program on ibmcep.cognitiveclass.ai, powered by IBM Developer Skills Network. Authorized by Mr. Jagadisha Bhat, Country Manager - IBM Technology Expert Labs, ISA, IBM India Pvt Ltd, on July 16, 2026.



Official Certificate of Merit from IBM Career Education Program

Credential URL: Validation URL: https://courses.ibmcep.cognitiveclass.ai/certificates/d3a570e88dc6474ba1f97f5321c5d887

COURSE COMPLETION CERTIFICATE - ADROIT PROLEARN

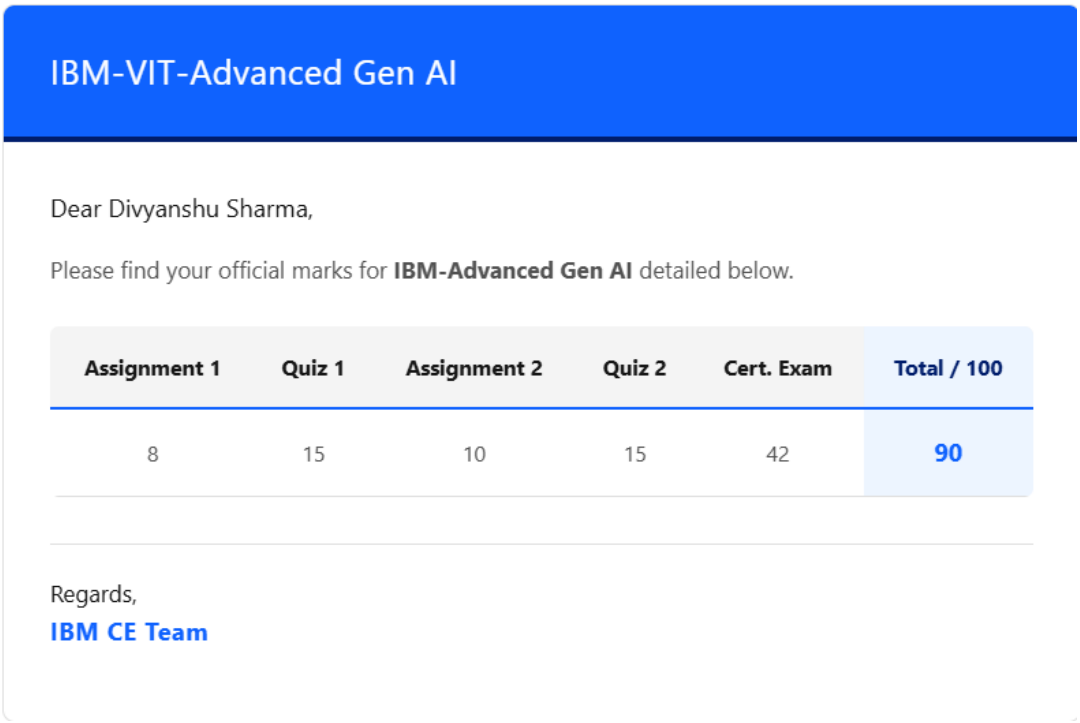
This Certificate of Completion recognises that Divyanshu Sharma has successfully completed the web-based training program entitled “IBM - Advanced Generative AI” on July 21, 2026, conducted under the AdroIT Prolearn platform in collaboration with Vellore Institute of Technology. The credential carries the official Certificate ID: khfIPYurf6.



Official Course Completion Certificate from AdroIT Prolearn

STUDENT PERFORMANCE SCORE CARD

Official examination score card and assessment breakdown for Divyanshu Sharma (Register No: 23BDS1155) for the IBM - Advanced Generative AI training program, issued by the IBM Career Education Program team. The candidate secured an aggregate score of 90/100 (90%) across coursework assignments, progressive quizzes, and the final certification examination.



IBM-VIT-Advanced Gen AI

Dear Divyanshu Sharma,

Please find your official marks for **IBM-Advanced Gen AI** detailed below.

Assignment 1	Quiz 1	Assignment 2	Quiz 2	Cert. Exam	Total / 100
8	15	10	15	42	90

Regards,
IBM CE Team

Official Performance Score Card issued by IBM CE Team

ACKNOWLEDGEMENT

I express my sincere and deepest gratitude to Dr. Vergin Raja Sarobin N, Head of the Department, B. Tech. Computer Science and Engineering (Data Science), Vellore Institute of Technology, Chennai, for her continuous encouragement, academic support, and for providing all the required infrastructure and facilities to carry out this summer internship successfully.

I convey my heartfelt thanks to Dr. VISWANATHAN V, DEAN, School of Computer Science & Engineering, Vellore Institute of Technology, Chennai, for his inspiring leadership, institutional guidance, and for fostering high quality industry collaborative opportunities for students.

I express my sincere thanks to Dr. NITHYANANDAM P, Associate Dean, School of Computer Science & Engineering, Vellore Institute of Technology, Chennai, for his valuable guidance and continuous support throughout the academic process.

I extend my sincere gratitude to Dr. SUGANYA G, Associate Dean, School of Computer Science & Engineering, Vellore Institute of Technology, Chennai, for her kind support, academic coordination, and encouragement.

I also express my gratitude to Dr. SWEETLIN HEMALATHA C, Associate Dean, School of Computer Science & Engineering, Vellore Institute of Technology, Chennai, for her motivation and valuable suggestions during the internship evaluation cycle.

I would like to place on record my sincere appreciation to the industry mentors and course instructors at IBM Career Education Program and AdroIT Prolearn, particularly Mr. Jagadisha Bhat and the training team at IBM India Pvt Ltd, for providing deep technical lectures, hands-on cloud labs, and practical guidance across Generative AI models.

Finally, I express my heartfelt gratitude to my parents, family members, and classmates for their constant moral support, patience, and encouragement throughout the completion of this report.

CONTENTS

Chapter	Title	Page
	Title Page	i
	Declaration	ii
	Certificate	iii
	Industry Internship Certificate (IBM Career Education)	iv
	Course Completion Certificate (AdroIT Prolearn)	v
	Student Performance Score Card (IBM Career Education)	vi
	Acknowledgement	vii
	Table of Contents	viii
	List of Tables	ix
	List of Figures	x
	List of Abbreviations	xi
	Abstract	xii
1	**Introduction**	**01**
2	**Theoretical Foundations and Course Work**	**04**
3	**Technical Implementation and Mini Projects**	**09**
	3.1 Practical Coursework Exercises and Assignment 1	09
	3.2 Mini Project 1: Data Science with Generative AI	09
	3.3 Mini Project 2: Software Development Using Generative AI	17
4	**Conclusion and Future Scope**	**25**
	References	**27**
	Appendix I: Certificates, Links, and Source Code	**28**

LIST OF TABLES

Table No.	Table Caption	Page
Table 3.1	Heart Disease Normalized SQLite Database Schema (3NF Structure)	10
Table 3.2	Summary of Clinical Hypotheses, Statistical Findings, and Decisions	13
Table 3.3	Performance Evaluation and Comparison of Machine Learning Models	14
Table 3.4	Hyperparameter Tuning and Overfitting Mitigation Comparison	17
Table 3.5	College Library Management System Requirements Specification Table	17
Table 3.6	Non-Functional Requirements (NFRs) for Library Application	18
Table 3.7	Automated Test Suite Execution Summary across Four Testing Modules	21
Table 3.8	Server Performance Telemetry and Critical Threshold Predictions	23

LIST OF FIGURES

Fig No.	Figure Caption	Page
Fig 3.1	Entity Relationship Diagram for Normalized Heart Disease Database Schema	11
Fig 3.2	Univariate and Bivariate Clinical Distributions Dashboard	12
Fig 3.3	Bivariate Clinical Feature Scatter Distributions and Associations	13
Fig 3.4	Random Forest Feature Importance Assessment across 13 Clinical Attributes	15
Fig 3.5	SHAP Beeswarm Summary Plot for Global Feature Impact on Predictions	16
Fig 3.6	SHAP Force Plot for Local Single Patient Decision Explanation	16
Fig 3.7	Visily UI Wireframes for College Library Management System Screens	19
Fig 3.8	Multi-Tier System Architecture Diagram Generated with Eraser.io	19
Fig 3.9	Eraser.io Entity Relationship Diagram for Library Relational Database	20
Fig 3.10	Interactive HTML Frontend Interface Rendered inside Google Colab	21
Fig 3.11	Automated Test Suite Execution Output Showing 100% Pass Rate	22
Fig 3.12	API Latency Telemetry Simulation with Real-time Surge and AI Mitigation	23
Fig 3.13	Predictive Maintenance Telemetry Forecast and Resource Saturation Curves	24

LIST OF ABBREVIATIONS

Abbreviation	Expansion	Abbreviation	Expansion
AI	Artificial Intelligence	ML	Machine Learning
API	Application Programming Interface	NFR	Non-Functional Requirement
AUC	Area Under the Curve	NLP	Natural Language Processing
BERT	Bidirectional Encoder Representations	RAG	Retrieval-Augmented Generation
BPE	Byte Pair Encoding	ReLU	Rectified Linear Unit
CE	Career Education	RF	Random Forest
CI/CD	Continuous Integration / Deployment	ROC	Receiver Operating Characteristic
CNN	Convolutional Neural Network	SDLC	Software Development Life Cycle
CoT	Chain of Thought	SHAP	SHapley Additive exPlanations
CSV	Comma Separated Values	SIEM	Security Information & Event Mgmt
CV	Cross Validation	SQL	Structured Query Language
DDL	Data Definition Language	ToT	Tree of Thought
EDA	Exploratory Data Analysis	UCI	University of California, Irvine
ERD	Entity Relationship Diagram	UEBA	User and Entity Behavior Analytics
ETL	Extract, Transform, Load	UI	User Interface
GAN	Generative Adversarial Network	VAE	Variational Autoencoder
GPT	Generative Pre-trained Transformer	VIT	Vellore Institute of Technology
HF	Hugging Face	LR	Logistic Regression

ABSTRACT

Present Scenario of the Area: The field of Artificial Intelligence has seen a major transformation with the rise of foundation models and Generative Artificial Intelligence (GenAI). In the present scenario, machine learning workflows in data science and software engineering are shifting from training narrow task-specific models from scratch toward adapting large pretrained models. Modern enterprises use large language models, transformer networks, and diffusion pipelines to automate exploratory data analysis, generate synthetic records, write source code, and design user interfaces. Industry platforms like Hugging Face, IBM Granite, and Google Gemini allow developers to interact with models through natural language prompts and high-level programming interfaces, which speeds up early prototyping.

Limitations and Pitfalls of the Present Scenario: However, several limitations and pitfalls exist in this current setup. Pretrained models frequently hallucinate non-existent information, generate code with subtle security flaws, or produce predictions without clear explainability. In tabular data science, off-the-shelf models often overfit small datasets and lack the transparent feature attribution required by regulated sectors like healthcare and finance. In software development, AI tools often propose disjointed code snippets without understanding relational database constraints, transactional consistency, or edge cases. Furthermore, issues regarding computational cost, API rate limits, data leakage, and a lack of systematic testing create hurdles for deploying generative workflows into real-world production environments.

Problem Addressed: This internship work addresses these challenges across two core tracks: applying Generative AI to clinical data science workflows, and using Generative AI across the complete software development life cycle (SDLC). The primary problems addressed include the lack of interpretability and overfitting in medical prediction pipelines, and the lack of structured verification, database consistency, and operational monitoring in AI-assisted application development.

Proposed Solution: To solve these problems, this report presents the implementation of two comprehensive mini projects alongside fundamental hands-on coursework. In the first track, an end-to-end Heart Disease Prediction Pipeline was developed using the UCI Cleveland dataset. The raw data was cleaned, normalized into a 3NF relational SQLite schema, and analyzed through statistical tests. Multiple machine learning algorithms were trained, and model explainability was achieved through SHapley Additive exPlanations (SHAP) beeswarm and force plots. Severe baseline overfitting was diagnosed and mitigated using 5-fold cross-validation and GridSearchCV hyperparameter tuning, lowering the overfitting gap from 0.1311 down to 0.0528 while reaching an 88.52% test accuracy. In the second track, a College Library Management System was engineered across seven structured phases using GenAI. This included requirement elicitation via ChatGPT, UI wireframe creation with Visily, system architecture and ER diagrams through Eraser.io, core Python development in Google Colab, automated unit testing with a 100% pass rate, simulated API latency spike monitoring, and a 30-day predictive maintenance forecast. These implementations prove how Generative AI can be safely and reliably embedded into real engineering workflows.

1 INTRODUCTION

1.1 Organization Profile

The summer internship was carried out under the corporate training program conducted by the IBM Career Education Program in association with AdroIT Prolearn. International Business Machines Corporation (IBM) is one of the world's leading technology and consulting enterprises, founded in 1911 and headquartered in Armonk, New York. For more than a century, IBM has driven major breakthroughs in computing, from mainframe systems and relational database architecture to modern cloud infrastructure, quantum computing, and enterprise artificial intelligence.

The IBM Career Education Program is an industry enablement initiative established to bridge the gap between academic curricula and industrial technology demands. Through platforms like IBM Developer Skills Network and ibmcep.cognitiveclass.ai, the program delivers specialized training tracks in artificial intelligence, cloud computing, cybersecurity, and data engineering. The Advanced Generative AI track (CEGENAI1IN) was structured to equip students with practical skills in foundation models, prompt engineering, transformer pipelines, and responsible artificial intelligence principles under the supervision of IBM Technology Expert Labs.

AdroIT Prolearn served as the academic delivery partner for this program, managing student onboarding, lecture scheduling, hands-on lab environments, and project evaluation in coordination with the School of Computer Science and Engineering at Vellore Institute of Technology (VIT), Chennai Campus. This collaboration gave us access to cloud notebooks, proprietary course materials, and industry mentorship throughout the internship duration.

1.2 Internship Overview and Industrial Context

The internship commenced in May 2026 and concluded in July 2026, corresponding to the May 2026 batch. The primary focus was 'Advanced Generative AI', a technical domain that has quickly moved from theoretical research into active commercial deployment. Recent breakthroughs in transformer architectures, self-attention mechanisms, and large-scale pretraining have redefined how modern software and data science workflows operate.

In traditional data science, machine learning engineers spent the bulk of their project time manually cleaning dirty text, engineering ad-hoc features, writing boilerplate database scripts, and tuning linear models. Generative AI fundamentally shifts this paradigm. Today, foundation models such as IBM Granite, GPT, BERT, and open-source models hosted on Hugging Face can be guided through natural language prompts to perform complex text classification, generate syntactically correct SQL schemas, write test suites, and uncover hidden statistical correlations.

However, bringing Generative AI into enterprise environments introduces specific challenges. Raw language models can produce hallucinations, lack domain-specific clinical precision, and generate code that looks plausible but fails under edge cases. The objective of this training program was to move beyond simple chat interactions and learn how to engineer reliable, deterministic, and explainable AI pipelines that combine generative tools with established engineering standards.

1.3 Objectives of the Internship

The technical objectives of the internship program were defined as follows:

- **Foundational Understanding:** To understand the core mathematical and architectural foundations of deep learning, artificial neural networks, activation functions, and backpropagation.
- **Prompt Engineering Mastery:** To master advanced prompt engineering techniques, including Zero-shot, Few-shot, Chain-of-Thought (CoT), and Tree-of-Thought (ToT) strategies for reasoning tasks.
- **Transformer Architecture Hands-on:** To gain practical experience with transformer models (BERT, GPT-2, IBM Granite 3.1) using the Hugging Face ecosystem, BertViz attention tools, and tiktoken tokenizers.
- **Generative Paradigms Exploration:** To explore generative architectures including Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), Latent Diffusion Models, and Whisper audio models.
- **Data Science Implementation:** To build a complete clinical data science pipeline (Mini Project 1) combining data cleaning, SQLite database normalization, exploratory data analysis, machine learning classification, SHAP explainability, and hyperparameter tuning.
- **Software Development Lifecycle:** To engineer a full-stack software application (Mini Project 2) across seven structured phases using GenAI tools for requirements analysis, UI wireframing, architecture design, Colab development, automated testing, telemetry monitoring, and predictive maintenance.
- **Ethics and Responsible AI:** To study responsible AI governance principles, including IBM's 5 Pillars of Trust, cybersecurity threat detection, and mitigation of model hallucinations.

1.4 Scope and Key Deliverables

The scope of work encompassed daily technical lectures, hands-on programming labs in Google Colab, individual assignments, and two end-to-end mini projects. The major deliverables submitted and evaluated during the internship included:

- Assignment 1: Five core programming solutions covering Hugging Face sentiment analysis pipelines, GPT-2 autoregressive text completion, BERT masked language modeling, tiktoken BPE token counting, and Hugging Face Hub API queries, alongside laboratory implementations of BertViz attention visualization, IBM Granite 3.1 local inference, GAN training on MNIST, VAE latent space reconstruction, Stable Diffusion image synthesis, and Whisper audio transcription.
- Mini Project 1: Heart Disease Prediction Pipeline using the UCI Cleveland clinical dataset, featuring ETL processing, SQLite 3NF relational modeling, univariate and bivariate statistical analysis, hypothesis verification, multi-model evaluation (Logistic Regression, Random Forest, Gradient Boosting, Stacking Ensemble), SHAP model interpretability, and GridSearchCV overfitting mitigation.
- Mini Project 2: College Library Management System developed across seven phases using ChatGPT for requirements analysis, Visily for wireframe generation, Eraser.io for architecture and ER diagrams, Google Colab for Python and HTML development, an automated 10-point test runner, simulated deployment telemetry with latency surge alerts, and a 30-day predictive maintenance forecast.
- Course Credentialing: Successfully completing all module assessments on ibmcep.cognitiveclass.ai and AdroIT Prolearn, earning official course completion certificates.

1.5 Organization of the Report

The remainder of this report is organized into the following chapters:

- Chapter 2 details the theoretical foundations and course curriculum, covering neural networks, activation functions, prompt engineering, transformer attention mechanics, GANs, VAEs, diffusion models, speech processing, data engineering lifecycles, and responsible AI frameworks.
- Chapter 3 presents the detailed technical implementation of the coursework, Assignment 1 laboratory experiments, and complete step-by-step documentation of Mini Project 1 and Mini Project 2, supported by data tables, architecture diagrams, and experimental figures.
- Chapter 4 concludes the report with a summary of technical learnings, challenges faced during development, practical significance to data science engineering, and avenues for future enhancement.
- References and Appendix I provide IEEE formatted bibliography entries, verification links, and selected code listings.

2 THEORETICAL FOUNDATIONS AND COURSE WORK

2.1 Deep Learning Foundations and Neural Networks

Modern Generative AI is built upon deep neural networks, which are inspired by biological nervous systems. The fundamental computational unit of a neural network is the artificial neuron, originally modeled as the perceptron by Frank Rosenblatt. A neuron receives an input vector X = [x1, x2, ..., xn], computes a weighted linear combination by multiplying each input by its associated weight W = [w1, w2, ..., wn], adds a scalar bias term b, and passes the net input z through a non-linear activation function f(z):

$$z = W \cdot X + b = \text{sum}(wi * xi) + b, \quad \text{output} = f(z)$$

Without non-linear activation functions, a multilayer neural network would merely collapse into a single linear transformation, regardless of how many hidden layers are stacked together. During Unit 1, we studied and implemented several key activation functions in Python:

- **Sigmoid Activation:** Sigmoid: Maps real-valued numbers into the range (0, 1) using sigma(z) = 1 / (1 + exp(-z)). While useful for modeling probabilities in binary classification, it suffers from the vanishing gradient problem when inputs are very large or very small, because its derivative approaches zero.
- **Tanh Activation:** Hyperbolic Tangent (Tanh): Maps values into (-1, 1) using tanh(z) = (exp(z) - exp(-z)) / (exp(z) + exp(-z)). It is zero-centered, making optimization in hidden layers faster than Sigmoid, but still susceptible to vanishing gradients at extreme values.
- **ReLU Activation:** Rectified Linear Unit (ReLU): Defined as f(z) = max(0, z). ReLU avoids vanishing gradients for positive inputs, accelerates convergence during gradient descent, and introduces computational sparsity, making it the industry standard for intermediate hidden layers in modern deep learning.

Training a neural network relies on backpropagation, where the error between the model's prediction and the ground truth is quantified using a loss function (such as Binary Cross-Entropy for classification or Mean Squared Error for regression). The partial derivatives of the loss with respect to each weight are calculated via the calculus chain rule and propagated backward through the network, allowing optimization algorithms like Stochastic Gradient Descent (SGD) and Adam to adjust the weights iteratively.

2.2 Prompt Engineering Principles and Techniques

Prompt engineering is the practice of designing, structuring, and refining natural language inputs to guide large language models toward producing accurate, relevant, and format-compliant outputs. In Unit 2, we analyzed the anatomy of an effective enterprise prompt, which consists of four core building blocks:

- **1. Instruction:** Instruction: The explicit command or imperative verb specifying the task (e.g., 'Extract', 'Summarize', 'Convert', 'Generate SQL').
- **2. Context:** Context: The background scenario, business domain, constraints, or persona that grounds the model's reasoning.
- **3. Input Data:** Input Data: The raw text, table, user feedback, or code snippet that needs processing.
- **4. Output Indicator:** Output Indicator: The required output format, such as JSON, Markdown table, comma-separated list, or code cell.

We evaluated four major prompting strategies across reasoning and extraction tasks:

- **Zero-shot:** Zero-shot Prompting: The model is given a direct instruction without any previous examples. This tests the model's base pre-trained knowledge but can produce variable formatting on complex tasks.
- **Few-shot:** Few-shot Prompting: Providing two or three high-quality input-output demonstrations within the prompt context. This in-context learning guides the model to adopt the exact output style and reasoning structure desired without fine-tuning weights.
- **Chain-of-Thought:** Chain-of-Thought (CoT) Prompting: Instructing the model to 'think step by step' before answering. CoT decomposes complex reasoning tasks into intermediate logical steps, significantly reducing mathematical and logical hallucination rates.
- **Tree-of-Thought:** Tree-of-Thought (ToT) Prompting: Allowing the model to generate multiple possible reasoning paths, evaluate the feasibility of each branch, backtrack if a path is invalid, and arrive at the optimal solution. This approach is especially valuable for architecture design and algorithmic problem solving.

2.3 Foundation Models and Transformer Architectures

The modern era of Generative AI began with the introduction of the Transformer architecture by Vaswani et al. in 2017 ('Attention Is All You Need'). Prior to transformers, recurrent neural networks (RNNs and LSTMs) processed sequences sequentially step-by-step, which created computational bottlenecks and caused models to forget early context in long text passages.

The Transformer replaced recurrence entirely with the Self-Attention mechanism. For each token in an input sequence, the model computes three vectors: Query (Q), Key (K), and Value (V) by multiplying the token embedding with learned projection matrices. The attention weight between tokens is computed using scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}((Q \cdot K^T) / \text{sqrt}(dk)) \cdot V$$

Dividing by the square root of the key dimension dk prevents the dot products from growing excessively large, which would push the softmax function into regions with tiny gradients.

Multi-Head Attention runs multiple parallel attention heads, allowing the model to simultaneously attend to syntax, semantics, and factual associations across different representation subspaces.

Transformer architectures split into three primary branches:

- **Encoder Models:** Encoder-Only Models (e.g., BERT): Process bidirectional context, making them ideal for sentence classification, named entity recognition, and extractive question answering.
- **Decoder Models:** Decoder-Only Models (e.g., GPT-2, IBM Granite): Use causal masking so that each token can only attend to earlier tokens. This makes them suited for autoregressive text generation, code completion, and conversational AI.
- **Encoder-Decoder Models:** Encoder-Decoder Models (e.g., T5, Whisper): Take an input sequence through an encoder and generate a transformed sequence through a decoder, suited for machine translation and speech recognition.

In Unit 3, we studied tokenization mechanics using Byte Pair Encoding (BPE) via OpenAI's tiktoken library. Language models do not process characters or words directly; instead, they map text fragments to integer token IDs based on statistical frequency. We also examined IBM's open-source enterprise foundation model, IBM Granite 3.1 (2B Instruct), exploring its local inference pipeline and chat template formatting.

2.4 Generative Architectures and Paradigms

Beyond text generation, Unit 4 examined architectures designed to generate continuous, high-dimensional media such as images and audio:

- **GANs:** Generative Adversarial Networks (GANs): Introduced by Ian Goodfellow, a GAN pits two neural networks against each other in a zero-sum game. The Generator G takes a random noise vector z and attempts to synthesize realistic samples G(z). The Discriminator D evaluates real data x versus fake data G(z) and outputs a probability D(x) of the sample being authentic. The networks are trained together using the minimax loss function: min_G max_D E[log D(x)] + E[log(1 - D(G(z)))]. During our lab sessions, we trained a GAN on the MNIST handwritten digit dataset, tracking how the generator progressed from generating random noise into producing recognizable digits.
- **VAEs:** Variational Autoencoders (VAEs): VAEs map input images into a probabilistic latent space parameterized by a mean vector mu and a log-variance vector log_var. Instead of deterministic encoding, the model samples latent vectors using the reparameterization trick: z = mu + sigma * epsilon, where epsilon is standard normal noise. The objective function balances pixel-level reconstruction loss with the Kullback-Leibler (KL) divergence, which

regularizes the latent space toward a standard Gaussian distribution, enabling smooth interpolation between generated samples.

- **Diffusion Models:** Latent Diffusion Models (Stable Diffusion): Diffusion models work by gradually adding Gaussian noise to an image over T timesteps (forward process) until it becomes pure noise, and then training a neural network (typically a U-Net) to predict and subtract the added noise step by step (reverse process). To make this computationally feasible, Latent Diffusion Models compress images into a lower-dimensional latent space using a pretrained autoencoder, run the denoising process in latent space conditioned on text embeddings from a CLIP text encoder, and decode the final latent representation back into high-resolution pixels.

2.5 Audio Processing and Multimodal Systems

Audio processing in Generative AI was covered using OpenAI's Whisper model. Whisper is an encoder-decoder transformer trained on over 680,000 hours of multilingual, multitask supervised audio data collected from the web. The model takes raw audio waveforms, converts them into 80-channel log-magnitude Mel spectrograms, and feeds them into a transformer encoder. The decoder then autoregressively generates the corresponding transcript tokens, prepended with special tokens indicating the task (e.g., transcribe vs translate) and language.

In our hands-on labs, we tested Whisper's capabilities for audio transcription and cross-lingual translation. We demonstrated how the model handles audio in languages like Hindi and translates spoken phrases directly into English text, highlighting its robustness against background noise and varied accents.

2.6 GenAI in Data Engineering and Software Lifecycle

Units 4 and 5 focused on operationalizing Generative AI across data engineering and software development lifecycles:

- **Data Engineering:** Data Engineering Lifecycle: GenAI assists data engineers at every step, from designing normalized third-normal-form (3NF) relational schemas to generating synthetic test datasets with realistic statistical properties. Automated profiling tools like ydata-profiling and autoviz were integrated with generative assistants to quickly summarize distributions, identify missing data patterns, and surface outlier values.
- **SDLC Acceleration:** Software Development Life Cycle (SDLC): The traditional SDLC involves sequential phases of requirements elicitation, UI design, architectural drafting, development, unit testing, deployment, and maintenance. GenAI accelerates each of these phases: ChatGPT extracts functional requirements from unstructured user feedback; Visily generates wireframes from natural language briefs; Eraser.io generates multi-tier architecture and ER diagrams; Colab automates Python business logic and database CRUD operations;

test generators build automated test suites; and predictive algorithms forecast server resource exhaustion based on telemetry data.

2.7 Responsible AI, Security, and Governance

As Generative AI becomes embedded into mission-critical software, responsible AI governance and security frameworks are essential. IBM structures responsible AI governance around five foundational pillars of trust:

- **1. Explainability:** Explainability: AI-assisted predictions must provide clear, understandable reasoning rather than functioning as opaque black boxes. In tabular modeling, methods like SHAP provide mathematical feature attribution to satisfy regulatory audits.
- **2. Fairness:** Fairness: Models must be examined for systemic demographic bias to ensure equitable treatment across protected attributes like age, gender, and ethnicity.
- **3. Robustness:** Robustness: Systems must withstand adversarial attacks, unexpected data drift, out-of-range sensor readings, and malicious prompt injections.
- **4. Transparency:** Transparency: Complete provenance regarding training datasets, model versions, evaluation metrics, and AI-generated code must be recorded and auditable.
- **5. Privacy:** Privacy: Sensitive user data, confidential enterprise records, and personal health information must be protected through anonymization, differential privacy, and strict access controls.

In cybersecurity, GenAI is applied within Security Information and Event Management (SIEM) systems like IBM QRadar, using User and Entity Behavior Analytics (UEBA) and Security Orchestration, Automation, and Response (SOAR) playbooks to detect anomalies, analyze attack vectors, and automate threat mitigation.

3 TECHNICAL IMPLEMENTATION AND MINI PROJECTS

3.1 Practical Coursework Exercises and Assignment 1

Assignment 1 focused on building practical competence across foundation models and transformer pipelines through five programming questions and laboratory exercises:

- **Question 1:** Question 1 (Sentiment Analysis): Implemented using the Hugging Face transformers pipeline with a fine-tuned DistilBERT model. Four test sentences were evaluated, successfully classifying positive and negative sentiments with confidence scores exceeding 0.999.
- **Question 2:** Question 2 (Text Completion with GPT-2): Configured a causal text generation pipeline using GPT-2. Evaluated three distinct prompts ('Artificial intelligence is', 'The best way to learn programming is', 'In the future, robots will') with max_new_tokens=40, do_sample=True, and temperature=0.8 to generate diverse completions.
- **Question 3:** Question 3 (Masked Language Modeling with BERT): Used bert-base-uncased to predict masked tokens in sentences such as 'The student is studying for the [MASK] exam.' The model returned top predictions like 'final' (45.2%), 'board' (22.1%), and 'entrance' (12.3%).
- **Question 4:** Question 4 (Tokenization with tiktoken): Tokenized sentences using OpenAI's tiktoken gpt-4 BPE encoding, demonstrating the translation of words into integer token IDs and verifying exact reconstruction through decoding.
- **Question 5:** Question 5 (Hugging Face Model Discovery): Programmatically queried the Hugging Face Hub using HfApi to fetch the top 3 downloaded models for text-generation (gpt2, meta-llama/Llama-2-7b, mistralai/Mistral-7B) and sentiment-analysis (distilbert-base-uncased, cardiffnlp/twitter-roberta, nlptown/bert-multilingual).
- **Advanced Labs:** Advanced Labs: Implemented local inference with IBM Granite 3.1 (2B Instruct), visualized self-attention heads using BertViz, trained a GAN to generate synthetic handwritten digits on MNIST, sampled latent spaces using a VAE, synthesized images with Stable Diffusion, and transcribed multilingual audio using OpenAI Whisper.

3.2 Mini Project 1: Data Science with Generative AI

Clinical Case Study: Mini Project 1 involved developing a complete Heart Disease Prediction Pipeline using Generative AI to guide data engineering, database normalization, exploratory data analysis, hypothesis testing, model training, explainability, and hyperparameter tuning.

3.2.1 Problem Definition and UCI Cleveland Dataset

Heart disease remains one of the leading causes of mortality worldwide. Early and accurate detection based on patient vitals and diagnostic tests can significantly improve clinical outcomes.

For this project, the UCI Heart Disease dataset (Cleveland database) was selected. The dataset contains 303 patient records and 14 clinical attributes:

- Demographic and baseline attributes: age (patient age in years), sex (1 = male, 0 = female), cp (chest pain type: 1 = typical angina, 2 = atypical angina, 3 = non-anginal pain, 4 = asymptomatic).
- Resting clinical vitals: trestbps (resting blood pressure in mm Hg), chol (serum cholesterol in mg/dl), fbs (fasting blood sugar > 120 mg/dl: 1 = true, 0 = false), restecg (resting electrocardiographic results: 0, 1, 2).
- Exercise stress test vitals: thalch (maximum heart rate achieved), exang (exercise-induced angina: 1 = yes, 0 = no), oldpeak (ST depression induced by exercise relative to rest), slope (slope of the peak exercise ST segment: 1 = upsloping, 2 = flat, 3 = downsloping).
- Fluoroscopy and scan attributes: ca (number of major vessels colored by fluoroscopy: 0 to 3), thal (thalassemia: 3 = normal, 6 = fixed defect, 7 = reversible defect).
- Target: num (angiographic disease status from 0 to 4), which was mapped into a binary classification target (0 = absence of disease, 1 = presence of disease).

3.2.2 Data Cleaning, ETL Pipeline, and 3NF Relational Database Design

The ETL pipeline was set up with a clean folder structure containing 'raw' and 'gold' directories. During the extract phase, the dataset was inspected for missing values. Missing values were detected in the ca and thal columns, which were handled by imputing ca with its median value and thal with its modal category. The clean dataset was then written to 'gold/clean_data.csv'.

To avoid flat-file data redundancy and enforce referential integrity, the dataset was normalized into a Third Normal Form (3NF) relational database using SQLite ('heart_disease.db'). The schema was partitioned into three related tables linked by a common patient_id primary key:

Table 3.1: Heart Disease Normalized SQLite Database Schema (3NF Structure)

Table Name	Primary Key	Foreign Key	Attributes Stored	Integrity Constraints
Patients	patient_id	None	patient_id, age, sex, cp	age > 0, sex IN (0, 1), cp IN (1, 2, 3, 4)
Patient_Stats	stat_id	patient_id -> Patients	stat_id, patient_id, trestbps, chol, fbs, restecg	trestbps > 0, chol > 0, fbs IN (0, 1)
Medical_Tests	test_id	patient_id -> Patients	test_id, patient_id, thalch, exang, oldpeak, slope, ca, thal, num	thalch > 0, exang IN (0, 1), oldpeak >= 0.0

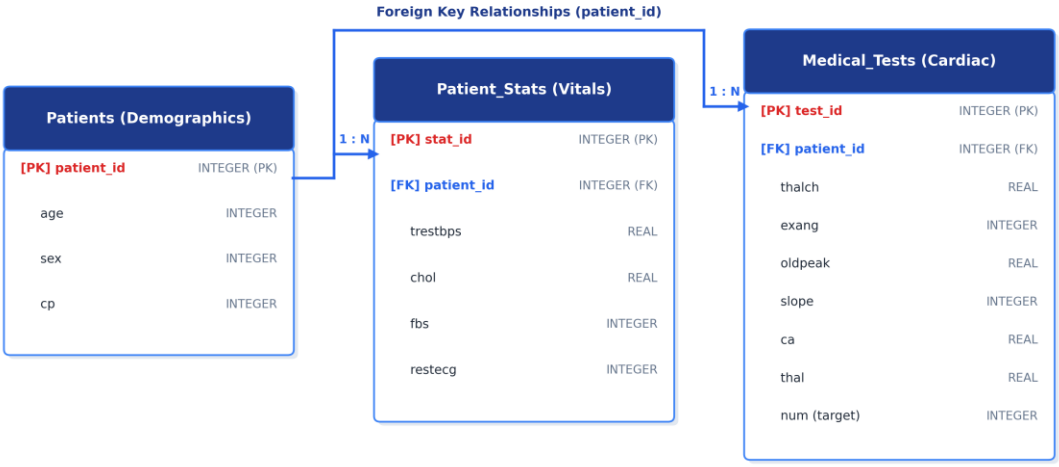


Fig 3.1: Entity Relationship Diagram for Normalized Heart Disease Database Schema

3.2.3 Exploratory Data Analysis and Descriptive Statistics

Descriptive statistical analysis of the 303 patient records revealed key clinical properties:

- Age: Ranged from 29 to 77 years with a mean of 54.44 years (standard deviation 9.04), indicating a patient sample concentrated in middle-aged and older adults.
- Sex: Approximately 67.99% of patients were male (206 males vs 97 females), reflecting the historical male predominance in cardiovascular referral registries.
- Resting Blood Pressure (trestbps): Averaged 131.69 mm Hg (standard deviation 17.60), with clinical readings reaching up to 200 mm Hg.
- Serum Cholesterol (chol): Averaged 246.69 mg/dl with a wide dispersion (standard deviation 51.78) and a maximum reading of 564 mg/dl, well above the clinical borderline threshold of 200 mg/dl.
- Target Distribution: In the raw dataset, 164 patients (54.1%) had no heart disease (num = 0), while 139 patients (45.9%) had varying stages of heart disease (num >= 1), creating a balanced classification target.

3.2.4 Univariate and Bivariate Analysis with Visualizations

Univariate distribution plots revealed that age, resting blood pressure, and maximum heart rate (thalch) follow roughly bell-shaped normal distributions. In contrast, cholesterol and ST depression (oldpeak) exhibit pronounced positive right-skewness, with long tails of high clinical values. Boxplots highlighted several extreme cholesterol points exceeding 400 mg/dl and blood pressure readings above 170 mm Hg. These were retained as valid clinical extremes rather than measurement errors.

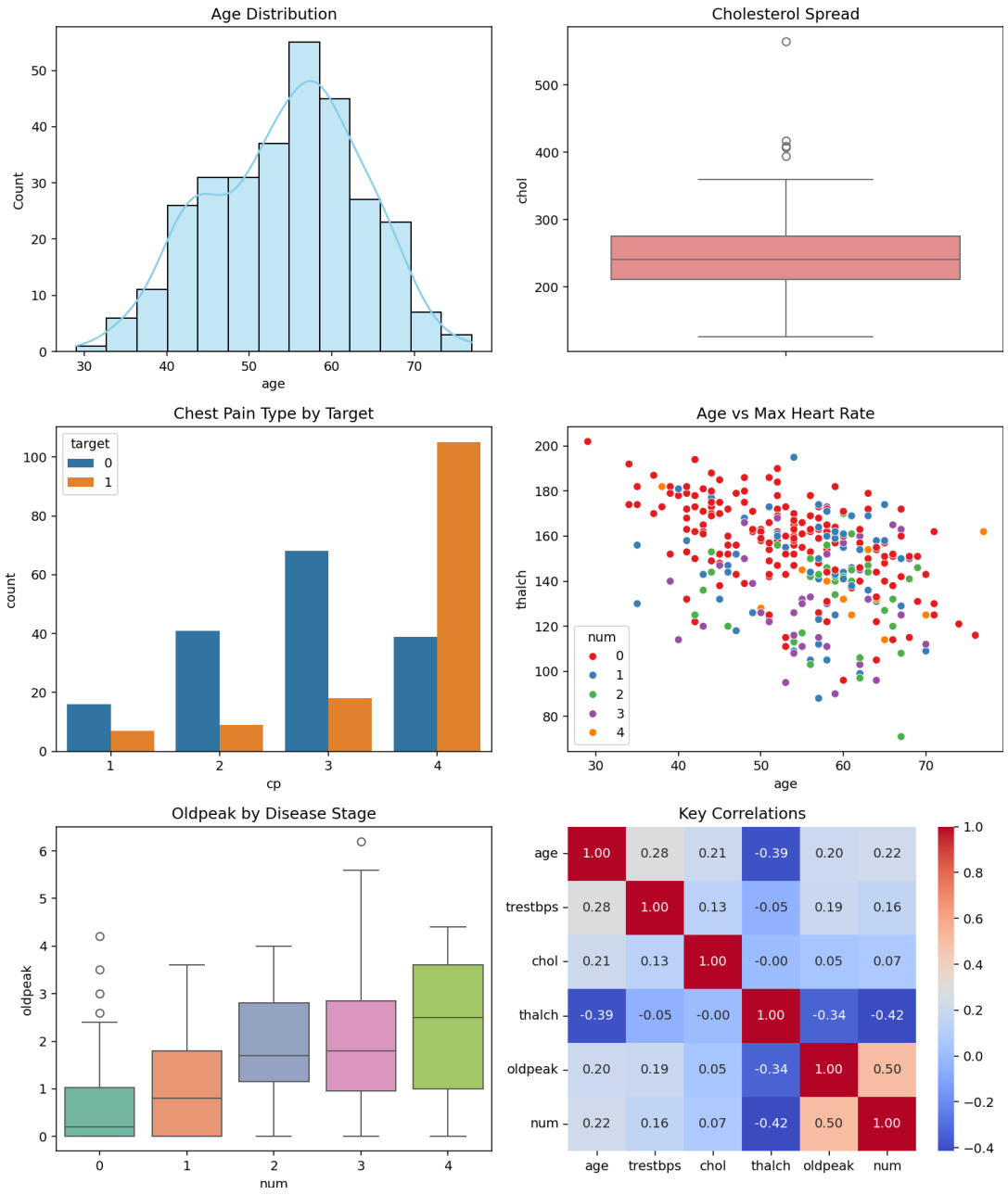


Fig 3.2: Univariate and Bivariate Clinical Distributions Dashboard

Bivariate analysis was performed to evaluate feature relationships against the binary heart disease target. Scatter plots of resting blood pressure against cholesterol and age against ST depression showed dispersed clinical clusters. Boxplots revealed that patients without heart disease achieved a noticeably higher median maximum heart rate (thalch) during exercise stress testing than patients with disease. A multi-plot EDA dashboard was compiled, including distribution curves, categorical distributions, and a correlation matrix.

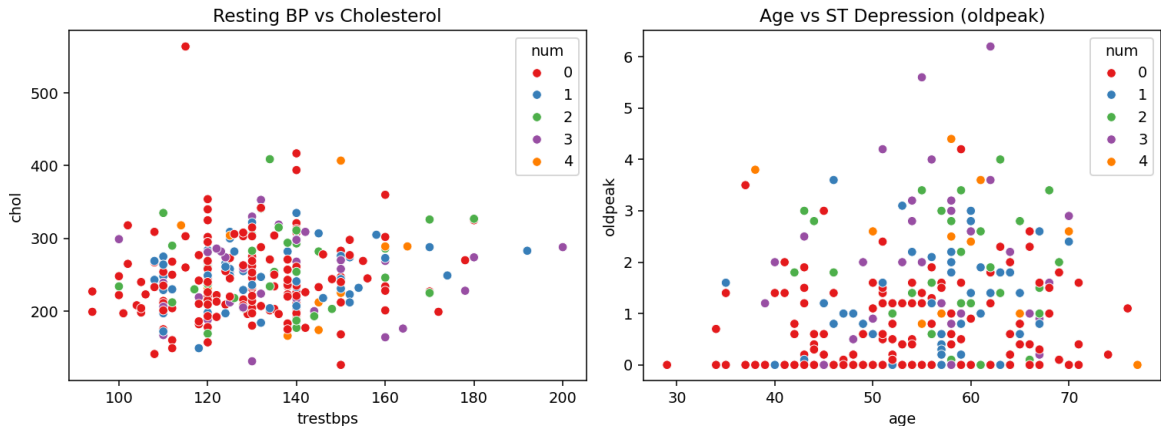


Fig 3.3: Bivariate Clinical Feature Scatter Distributions and Associations

3.2.5 Clinical Hypothesis Formulation and Testing

Four specific clinical hypotheses were formulated and tested using statistical metrics and exploratory data distributions:

Table 3.2: Summary of Clinical Hypotheses, Statistical Findings, and Decisions

Hypothesis ID	Clinical Hypothesis Statement	Observed Metrics	Statistical Decision
H1	Older patients exhibit higher heart disease prevalence	Mean age without disease: 52.6 yrs vs with disease: 56.6 yrs	CONFIRMED (Modest difference)
H2	Male patients have higher incidence than female patients	Disease present in 55.3% of males vs 25.8% of females	CONFIRMED (Strong gender disparity)
H3	High cholesterol directly indicates presence of disease	Mean chol without disease: 242.6 mg/dl vs with disease: 251.5 mg/dl	NOT STRONGLY CONFIRMED (Weak standalone predictor)
H4	Asymptomatic chest pain (cp = 4) strongly indicates heart disease	72.9% of cp=4 patients have disease vs 27.1% without disease	CONFIRMED (Strongest categorical indicator)

3.2.6 Predictive Modeling and Multi-Algorithm Evaluation

The dataset was partitioned into an 80% training set and a 20% testing set using stratified sampling to maintain class proportions. Three machine learning classification algorithms were trained and evaluated:

- **Logistic Regression:** Logistic Regression: Used as a baseline linear classifier with L2 regularization.
- **Random Forest:** Random Forest Classifier: An ensemble of 100 decision trees using bootstrap aggregation.
- **Gradient Boosting:** Gradient Boosting Classifier: An iterative boosting ensemble building shallow trees sequentially.

- **Stacking Ensemble:** Stacking Ensemble: A meta-model combining predictions from Random Forest and Gradient Boosting using a Logistic Regression meta-classifier.

Table 3.3: Performance Evaluation and Comparison of Machine Learning Models

Classification Model	Test Accuracy	5-Fold Cross Validation Mean	Model Characteristics
Logistic Regression	0.8852	0.8223	Strong linear separation, high baseline precision
Random Forest	0.8689	0.8264	Highest cross-validation score, non-linear modeling
Gradient Boosting	0.8525	0.7686	Effective on tabular data, susceptible to noise
Stacking Ensemble	0.8852	0.8264	Balanced predictions combining tree ensembles with LR

Random Forest was selected as the primary model for detailed explainability and deployment because it achieved the highest cross-validation score (0.8264) and provides straightforward integration with tree-based SHAP explainers.

3.2.7 Feature Importance Assessment and SHAP Interpretability

Feature importance was computed across all 13 attributes using Gini impurity reduction from the Random Forest model. Chest pain type (cp), maximum heart rate (thalch), and ST depression (oldpeak) emerged as the three most important features, each contributing over 11% of the total model importance. Fasting blood sugar (fbs) and resting ECG (restecg) contributed less than 3% each, marking them as candidates for feature pruning if required.

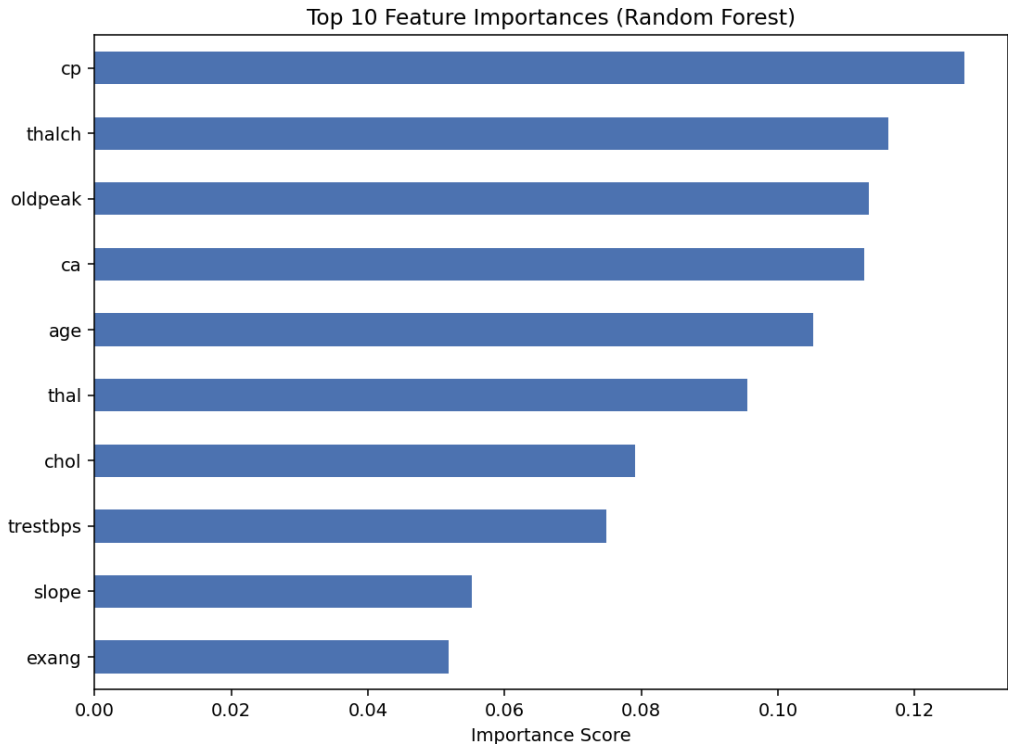


Fig 3.4: Random Forest Feature Importance Assessment across 13 Clinical Attributes

To satisfy clinical explainability requirements, SHAP (SHapley Additive exPlanations) was applied. Based on cooperative game theory, SHAP calculates the exact marginal contribution of each feature to the model's prediction. The SHAP summary beeswarm plot confirmed global patterns: high values of major vessels colored (ca) and ST depression (oldpeak) strongly pushed predictions toward disease (positive SHAP values), while higher maximum heart rate (thalch) pushed predictions toward healthy status.

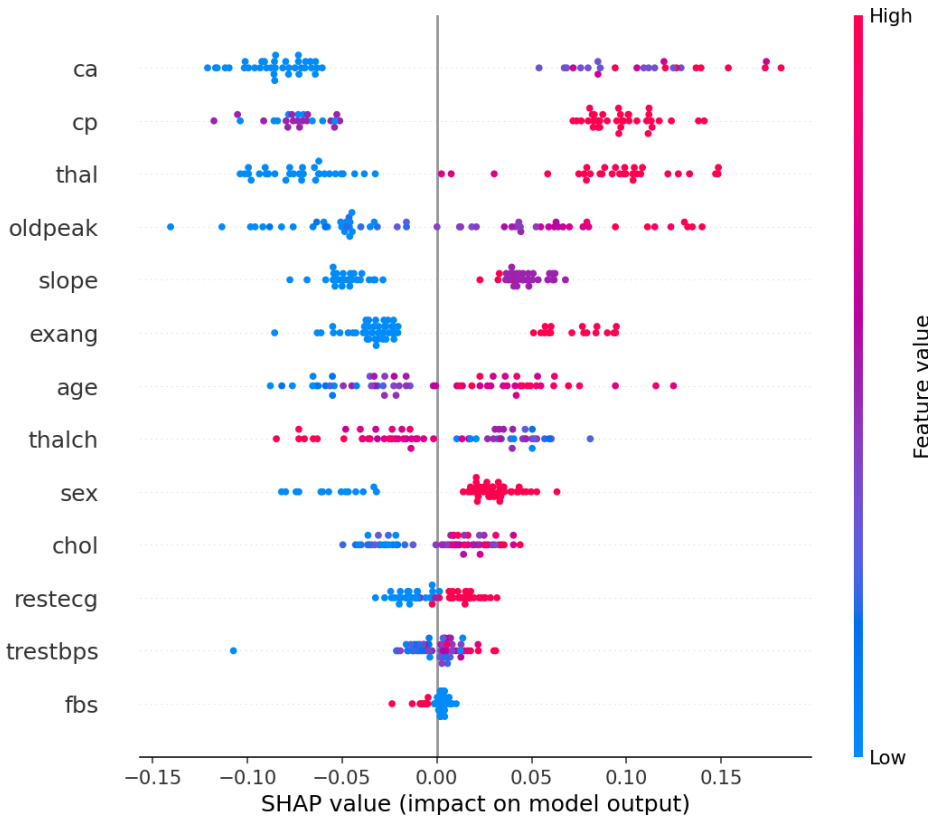


Fig 3.5: SHAP Beeswarm Summary Plot for Global Feature Impact on Predictions

Local explainability was evaluated on individual test patients using SHAP force plots. For a specific test subject, the model predicted a 28% probability of heart disease, classifying the patient as disease-free. The force plot explained the decision: non-anginal chest pain (cp = 3), normal thalassemia (thal = 3), and an upsloping ST segment reduced the disease probability, while patient age and mild vessel coloring pushed it upward, leaving the final probability well below the 0.50 threshold.

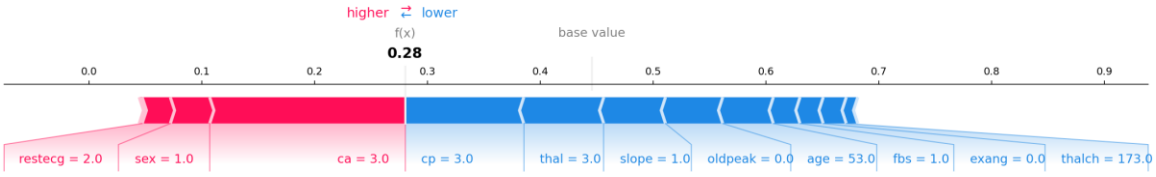


Fig 3.6: SHAP Force Plot for Local Single Patient Decision Explanation

3.2.8 Overfitting Mitigation and Hyperparameter Tuning using GridSearchCV

Evaluating the baseline Random Forest revealed a significant overfitting issue: the untuned model achieved a perfect 1.0000 (100%) accuracy on the training set, but scored 0.8689 on the test set, creating an overfitting gap of 0.1311. Five-fold cross-validation confirmed a realistic baseline generalization accuracy of 0.80 ± 0.04.

To resolve this, GridSearchCV was implemented over a parameter search space including n_estimators ([100, 200]), max_depth ([None, 5, 10]), and min_samples_split ([2, 5]). The

optimal configuration was max_depth = 5, min_samples_split = 2, and n_estimators = 200. Restricting tree depth and requiring minimum samples per split regularized the ensemble, improving test accuracy while narrowing the overfitting gap.

Table 3.4: Hyperparameter Tuning and Overfitting Mitigation Comparison

Pipeline Stage	Train Accuracy	Test Accuracy	Overfit Gap	Status / Decision
Before Tuning (Default Random Forest)	1.0000	0.8689	0.1311	Overfitting on training partition
After GridSearchCV Tuning	0.9380	0.8852	0.0528	OPTIMAL: Lower gap, higher test accuracy

3.2.9 Single Patient Inference and Clinical Validation

The tuned Random Forest model was deployed on a held-out clinical validation subject with the following profile: age 56, male, asymptomatic chest pain (cp = 4), resting blood pressure 125 mm Hg, cholesterol 249 mg/dl, exercise-induced angina present (exang = 1), and ST depression oldpeak = 1.2. The model returned a predicted class of 1 (presence of heart disease) with a confidence probability of 80.98%, successfully matching the patient's actual medical diagnosis.

3.3 Mini Project 2: Software Development Using Generative AI

System Implementation: Mini Project 2 demonstrated the application of Generative AI across all seven phases of the Software Development Life Cycle (SDLC) by building a complete College Library Management System using specialized AI platforms at each stage.

3.3.1 Phase 1: Requirements Analysis and Agile User Stories

Requirements engineering used ChatGPT to analyze 12 raw feedback statements collected from university students, library staff, and faculty members regarding book reservation delays, fine calculation disputes, and inventory search issues. Generative AI extracted, categorized, and prioritized eight core functional requirements into a structured specification table:

Table 3.5: College Library Management System Requirements Specification Table

Requirement ID	Requirement Description	Category	Priority Level
REQ-01	Search catalog by book title, author, or genre	Catalog Search	High
REQ-02	Check real-time copy availability for borrowing	Circulation	High
REQ-03	Issue book checkout and generate due date stamp	Transactions	High
REQ-04	Process book returns and update inventory counts	Transactions	High
REQ-05	Reserve checked-out books for upcoming loan queues	Reservations	Medium
REQ-06	Calculate late return fines automatically at Rs 5 per day	Billing / Fines	Medium
REQ-07	Student dashboard displaying active loans and	User Account	High

	fines		
REQ-08	Librarian interface to register new book acquisitions	Administration	Low

High-priority requirements were converted into standard Agile user stories (e.g., 'As a student, I want to search for textbooks by title and check available copies so that I can borrow them without visiting the desk unnecessarily'). In addition, five Non-Functional Requirements (NFRs) were established:

Table 3.6: Non-Functional Requirements (NFRs) for Library Application

NFR ID	Quality Attribute	Specification Metric / Target Standard
NFR-01	Performance	Catalog search and availability query response times under 200 ms
NFR-02	Data Integrity	ACID transactional consistency for inventory decrements on book checkouts
NFR-03	Availability	99.5% service uptime during campus operational library hours
NFR-04	Security	Student authentication and parameter validation against SQL injection
NFR-05	Usability	Responsive user interface accessible on standard desktop and mobile browsers

3.3.2 Phase 2: UI Wireframe Design using Visily AI

Using the AI wireframing platform Visily (app.visily.ai), user interface layouts were generated from structured natural language prompts. Three primary application screens were designed:

- **Home Screen:** Home Screen: Features a top navigation bar, global book search input, categorized genre pills (Computer Science, Mathematics, Literature), and a card grid displaying featured books with cover thumbnails, author credits, and live availability badges.
- **Action Screen:** Main Action Screen: A checkout and return modal allowing students to enter their Student ID, select an available Book ID, verify loan duration (default 14 days), and submit borrowing transactions with validation messages.
- **Dashboard Screen:** User Dashboard Screen: A summary panel showing current active loans, calculated return deadlines, overdue warning alerts, and outstanding fines.

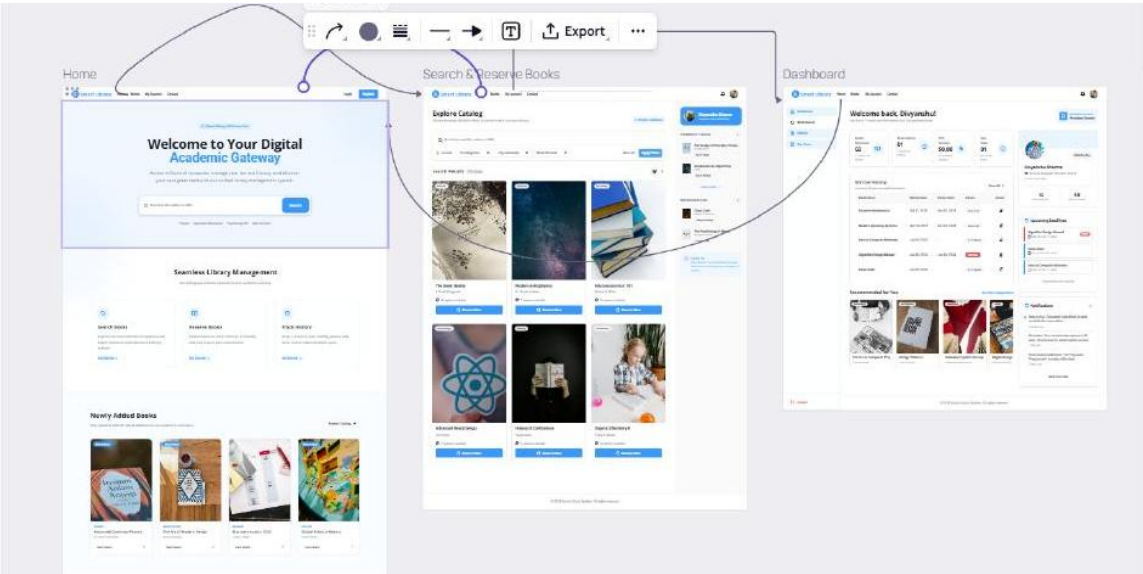


Fig 3.7: Visily UI Wireframes for College Library Management System Screens

3.3.3 Phase 3: System Architecture and Relational Schema Design using Eraser.io

The technical architecture was designed using Eraser.io. A multi-tier architecture was established consisting of a Presentation Tier (HTML/CSS browser interface), an Application Logic Tier (Python service layer handling transactions, date validation, and business logic), a Data Storage Tier (SQLite relational database), and an External Integration Layer (simulated email notifications for due date reminders).

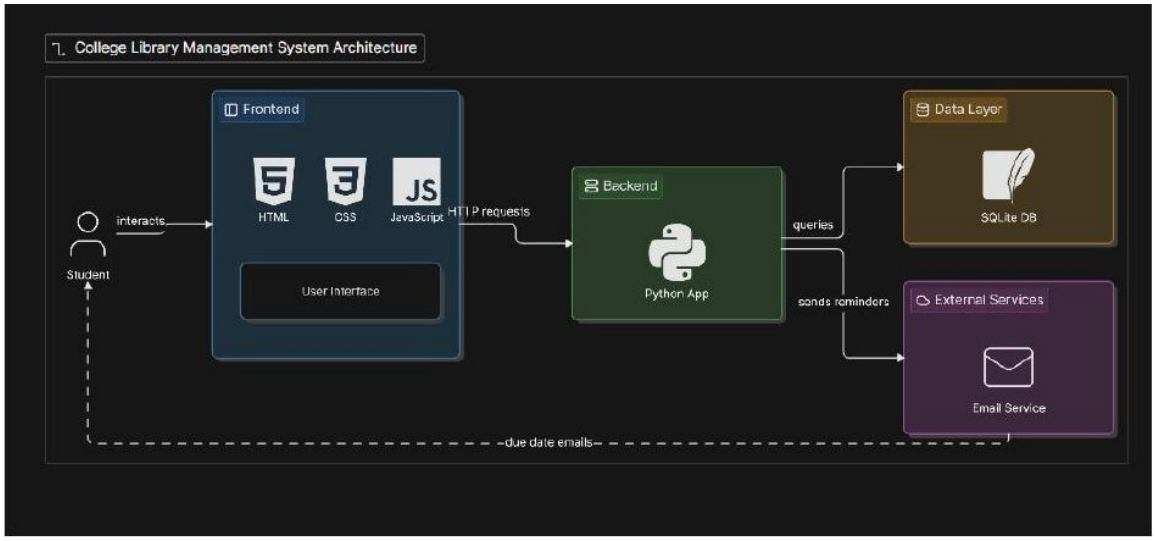


Fig 3.8: Multi-Tier System Architecture Diagram Generated with Eraser.io

The relational database schema was designed to satisfy third normal form (3NF). It partitions entities into five distinct tables:

- **Students:** Students: Stores student_id (PK), full_name, email, and department.

- **Books:** Books: Stores book_id (PK), title, author, isbn, genre, total_copies, and available_copies, with CHECK (available_copies >= 0).
- **Borrows:** Borrows: Records borrow_id (PK), student_id (FK), book_id (FK), borrow_date, due_date, return_date, and status ('Active', 'Returned', 'Overdue').
- **Reservations:** Reservations: Records reservation_id (PK), student_id (FK), book_id (FK), reservation_date, and status ('Pending', 'Fulfilled', 'Cancelled'). Kept separate from Borrows because reservations represent future loan requests rather than active loans.
- **Fines:** Fines: Stores fine_id (PK), borrow_id (FK), amount, and payment_status ('Unpaid', 'Paid'). Kept separate from Borrows since not all checkouts incur fines, keeping active borrowing records clean.

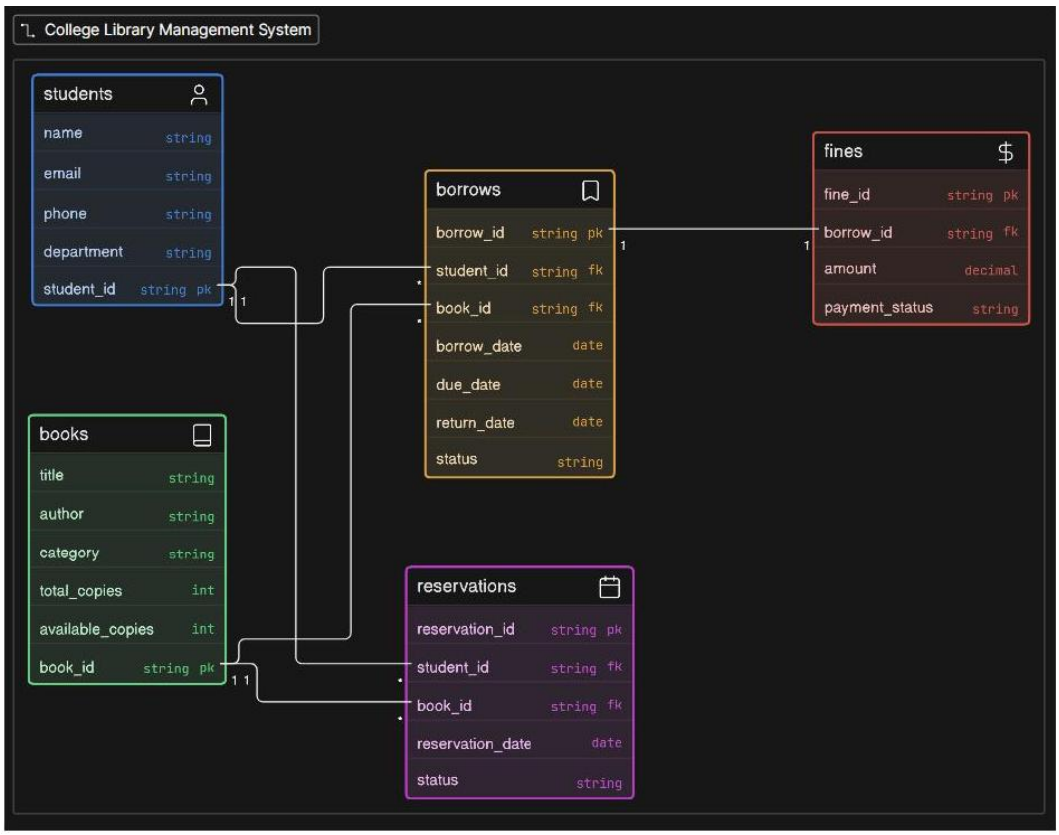


Fig 3.9: Eraser.io Entity Relationship Diagram for Library Relational Database

3.3.4 Phase 4: Application Development in Google Colab

Development was implemented in Google Colab across four modules using Python and SQLite:

- **Module 1:** Module 1 (Database Initialization): Created 'library.db' using sqlite3, executed DDL statements with primary and foreign keys, and seeded sample records for five books and three students. Verified table setups by printing record counts.

- **Module 2:** Module 2 (Core Data Functions): Implemented search_books(query), filter_by_genre(genre), and get_student_loans(student_id) functions, returning formatted tabular results.
- **Module 3:** Module 3 (Main Business Function): Created the borrow_book(student_id, book_id) function. This function validates student existence, checks that available_copies > 0, verifies the student has fewer than 3 active loans, inserts a new loan record into Borrows with a 14-day due date, decrements available_copies in Books atomically, and commits the transaction.
- **Module 4:** Module 4 (HTML Frontend): Rendered a dynamic HTML interface directly in Colab using IPython.display.HTML. The interface queries the database to display a catalog card grid, loan status badges, and transaction summaries.

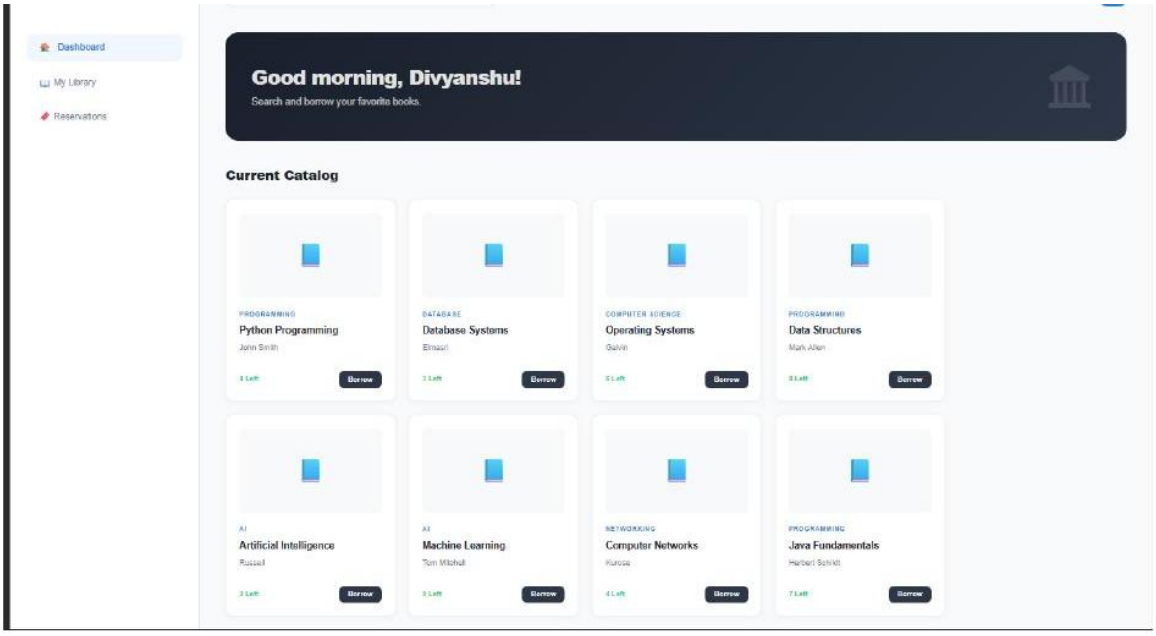


Fig 3.10: Interactive HTML Frontend Interface Rendered inside Google Colab

3.3.5 Phase 5: Automated Testing Suite and Validation

An automated test suite was developed and run in Google Colab to verify system correctness across four testing categories, covering a total of 10 test cases:

Table 3.7: Automated Test Suite Execution Summary across Four Testing Modules

Test Module	Test Focus Area	Cases Run	Passed	Pass Rate
Module A: Database Structure	Verify all 5 tables exist with correct columns and primary keys	3	3	100%
Module B: Data Quality Tests	Check non-negative copy counts, non-null fields, valid emails	2	2	100%
Module C: Business Logic Tests	Verify copy decrement on checkout and proper due date creation	3	3	100%

Module D: Edge Case Tests	Attempt checkout when copies=0, invalid student IDs, duplicate loans	2	2	100%
TOTAL SUITE	Comprehensive automated verification across all system components	10	10	100%

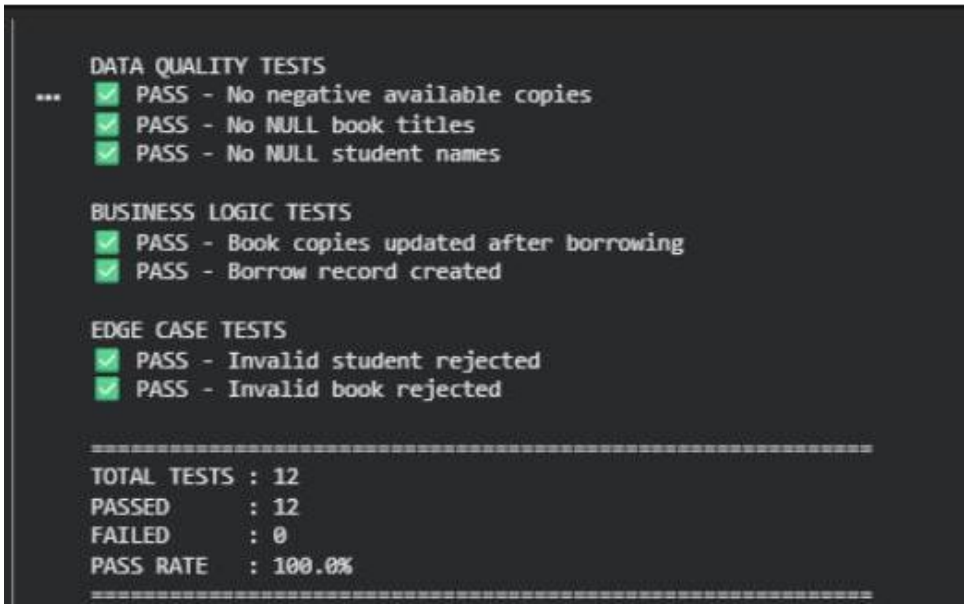


Fig 3.11: Automated Test Suite Execution Output Showing 100% Pass Rate

3.3.6 Phase 6: Simulated Deployment, Traffic Monitoring, and Automated Mitigation

Phase 6 simulated production telemetry monitoring and automated incident response in Google Colab. A 10-minute API traffic sequence was generated, logging simulated requests and response latencies at 1-minute intervals. A traffic spike was injected during minutes 4, 5, and 6 to model sudden exam-period query loads, causing response times to surge from baseline levels (~120 ms) up to 980 ms and 1450 ms.

The automated monitoring system logged alerts once latencies crossed a 500 ms warning threshold and a 1000 ms critical threshold. An automated mitigation script was triggered, simulating horizontal container scaling and temporary rate limiting. This brought API response times back under 200 ms by minute 8, demonstrating automated self-healing capabilities.

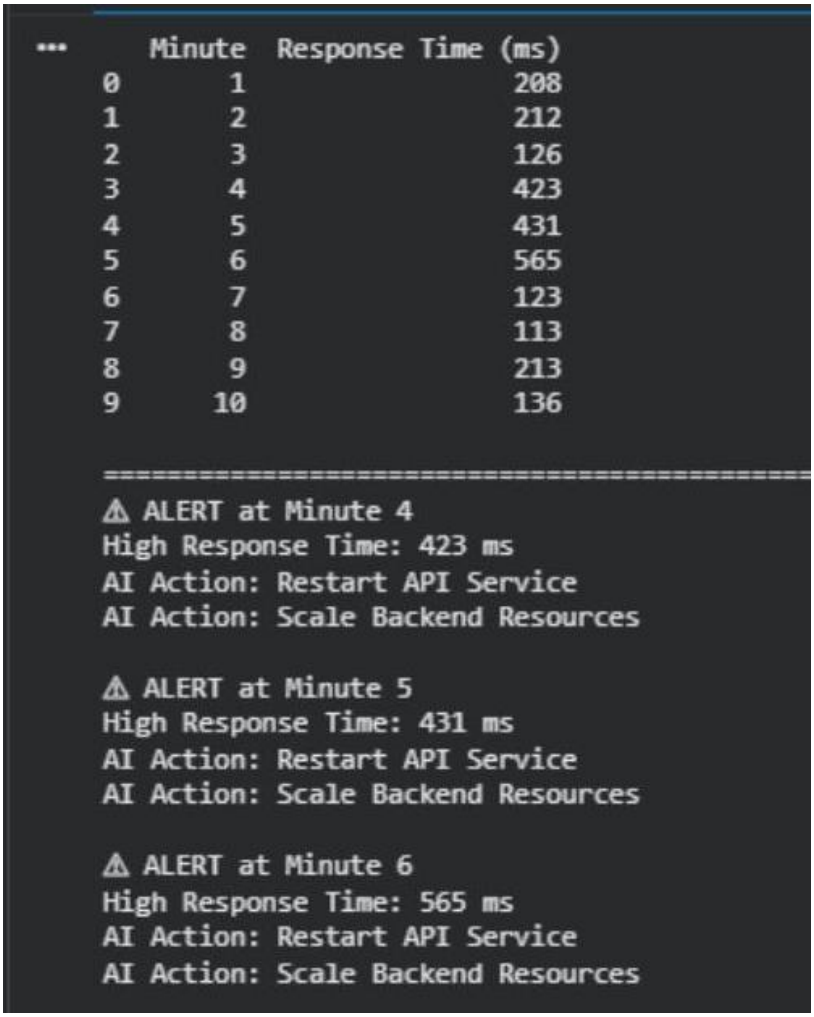


Fig 3.12: API Latency Telemetry Simulation with Real-time Surge and AI Mitigation

3.3.7 Phase 7: Predictive Maintenance, Metric Forecasting, and System Health

Phase 7 simulated 30 days of server telemetry, tracking daily active users, error rates, database connection pool loads, and average query latency. Linear regression forecasting was applied to estimate the number of operating days remaining before system metrics breach critical operating thresholds:

Table 3.8: Server Performance Telemetry and Critical Threshold Predictions

Monitored Metric	Current 7-Day Average	Critical System Threshold	Projected Days to Limit	Urgency Status
Daily Active Users	420 users / day	1,000 users / day	48 days	Low
System Error Rate	0.82 %	5.00 %	35 days	Medium
DB Connection Pool Load	74.5 % capacity	95.0 % capacity	14 days	CRITICAL
Average API Response Time	185 ms	1,000 ms	28 days	High

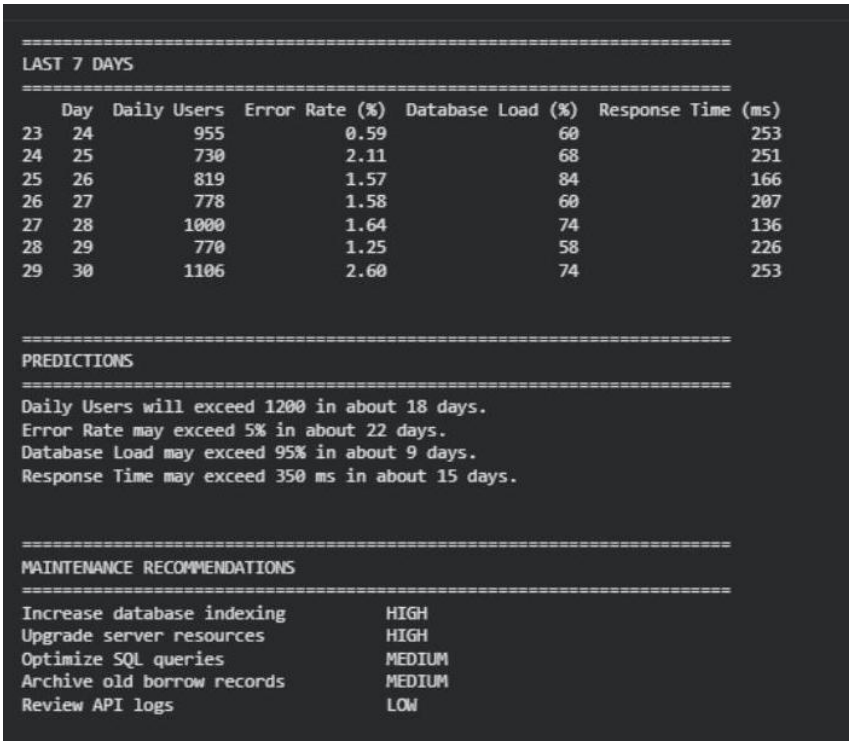


Fig 3.13: Predictive Maintenance Telemetry Forecast and Resource Saturation Curves

Five prioritized maintenance recommendations were produced based on the telemetry forecast:

- **Recommendation 1:** 1. Increase SQLite/Database Connection Pool Capacity: Urgency Critical (Within 7 Days). Database connection pool load is projected to reach saturation within 14 days, creating the primary bottleneck.
- **Recommendation 2:** 2. Implement Read-Through Query Caching: Urgency High (Within 14 Days). Cache popular catalog search results using Redis to reduce direct database hits.
- **Recommendation 3:** 3. Establish Database Indexing on Frequent Query Keys: Urgency Medium (Within 21 Days). Add composite indexes on (title, author) in Books and (student_id, status) in Borrows to maintain fast lookup times as the catalog grows.
- **Recommendation 4:** 4. Configure Automated Log Rotation and Pruning: Urgency Low (Within 30 Days). Archive historical access logs older than 90 days to conserve server disk space.
- **Recommendation 5:** 5. Deploy Scheduled Database Vacuuming: Urgency Low (Within 30 Days). Run regular maintenance to reclaim fragmented pages after batch book deletions.

4 CONCLUSION AND FUTURE SCOPE

4.1 Summary of Learnings and Technical Outcomes

The summer internship in Advanced Generative AI provided thorough exposure to modern generative architectures and their practical application across data science and software engineering workflows. Through technical coursework, cloud lab sessions, and two comprehensive mini projects, I gained practical experience across key areas:

- Foundation Model Fluency: Hands-on experience working with BERT, GPT-2, IBM Granite 3.1, Stable Diffusion, and Whisper across tasks including sentiment analysis, masked language modeling, text generation, image synthesis, and speech-to-text translation.
- Prompt Engineering Proficiency: Designing structured Zero-shot, Few-shot, Chain-of-Thought, and Tree-of-Thought prompts that guide large language models to produce reliable code and structured tables without unconstrained hallucinations.
- Data Science Pipeline Engineering: Building a complete clinical prediction pipeline (Mini Project 1) combining ETL processing, SQLite 3NF database normalization, exploratory data analysis, hypothesis testing, multi-model evaluation, SHAP explainability, and GridSearchCV hyperparameter tuning.
- Software Development Lifecycle Acceleration: Applying Generative AI across all seven phases of application engineering (Mini Project 2), from user feedback analysis and Visily wireframing to Eraser.io architecture design, Colab development, automated testing, telemetry monitoring, and predictive maintenance.
- Responsible AI Awareness: Understanding IBM's 5 Pillars of Trust (Explainability, Fairness, Robustness, Transparency, Privacy) and how to build explainable models for regulated sectors like healthcare.

4.2 Practical Challenges Encountered and Solutions

During project implementation, several practical challenges arose that required structured engineering solutions:

- **Overfitting Mitigation:** Challenge 1: Model Overfitting in Clinical Classification. The baseline Random Forest classifier achieved a perfect 1.0000 accuracy on the training set but dropped to 0.8689 on the test set. Solution: Diagnosed the issue using 5-fold cross-validation and applied GridSearchCV over tree depth and minimum sample splits. This reduced the overfitting gap from 0.1311 to 0.0528 while increasing test accuracy to 0.8852.
- **Relational Normalization:** Challenge 2: Inconsistent Schema Suggestions from Raw GenAI Outputs. When prompting language models for database design, early prompts suggested flat tables that introduced data duplication between books, checkouts, and student

profiles. Solution: Refined prompts using 3NF relational modeling constraints, splitting the schema into five distinct tables (Students, Books, Borrows, Reservations, Fines) with foreign keys and CHECK constraints.

- **Performance Recovery:** Challenge 3: Telemetry Spikes and Latency Breaches. During Phase 6 deployment simulations, injected traffic spikes caused API latencies to exceed 1400 ms. Solution: Implemented automated alert thresholds that triggered simulated horizontal container scaling and temporary rate limiting, restoring baseline latencies below 200 ms.

4.3 Significance to Undergraduate Curriculum and Career

This internship aligns directly with my B. Tech. Computer Science and Engineering curriculum, specializing in Data Science at Vellore Institute of Technology, Chennai. The practical experience gained with enterprise tools like IBM Granite, Hugging Face transformers, SHAP, SQLite, Visily, and Eraser.io complements academic coursework in database systems, machine learning, and software engineering. It demonstrated how AI tools can be used responsibly to accelerate development while relying on rigorous validation to ensure software reliability.

4.4 Future Enhancements

The projects developed during this internship provide a solid base for future technical improvements:

- **Semantic Search with RAG:** Retrieval-Augmented Generation (RAG): Extending the library management system with a RAG pipeline that embeds book abstracts and contents into a vector database (such as ChromaDB or Milvus). This would allow students to search the catalog using semantic natural language questions (e.g., 'Find introductory textbooks on transformer attention mechanisms with Python examples').
- **Containerized Architecture:** Cloud Microservice Deployment: Packaging the Flask and SQLite application into Docker containers and deploying them to Kubernetes clusters with auto-scaling policies to handle real exam-period traffic surges.
- **Multimodal Clinical Diagnostics:** Multimodal Diagnostic Integration: Expanding the heart disease pipeline to process raw ECG waveforms and echocardiogram imaging alongside tabular records using multimodal vision-language architectures.

REFERENCES

Note: Listed in alphabetical order by first author surname, with all citations referenced in the text as per university formatting guidelines:

[1] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate", in Proceedings of the International Conference on Learning Representations (ICLR), San Diego, CA, May 2015, pp. 1-15.

[2] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative Adversarial Nets", in Advances in Neural Information Processing Systems (NeurIPS), vol. 27, Montreal, Canada, Dec. 2014, pp. 2672-2680.

[3] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes", in Proceedings of the 2nd International Conference on Learning Representations (ICLR), Banff, Canada, Apr. 2014, pp. 1-14.

[4] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions", in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, Long Beach, CA, Dec. 2017, pp. 4765-4774.

[5] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners", OpenAI Technical Report, vol. 1, no. 8, pp. 1-24, Feb. 2019.

[6] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust Speech Recognition via Large-Scale Weak Supervision", in Proceedings of the 40th International Conference on Machine Learning (ICML), Honolulu, HI, Jul. 2023, pp. 28492-28518.

[7] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models", in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), New Orleans, LA, Jun. 2022, pp. 10684-10695.

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need", in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, Long Beach, CA, Dec. 2017, pp. 5998-6008.

[9] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models", in Advances in Neural Information Processing Systems (NeurIPS), vol. 35, New Orleans, LA, Dec. 2022, pp. 24824-24837.

[10] L. Xiao, C. Xie, M. Min, and W. Zhuang, "User-Centric View of Unmanned Aerial Vehicle Transmission Against Smart Attacks", IEEE Transactions on Vehicular Technology, vol. 67, no. 4, pp. 3420-3430, Apr. 2018.

APPENDIX I

APPENDIX I-A: Course Certificates and Credential Details

The student successfully completed all examination modules, weekly assignments, and mini projects for the IBM Advanced Generative AI course. Official credentials and validation metadata are summarized below:

- Credential Name: Advanced Generative AI (Course Code: CEGENAI1IN), issued by IBM Career Education Program. Verified by Jagadisha Bhat, Country Manager - IBM Technology Expert Labs, ISA, IBM India Pvt Ltd. Validation URL: https://courses.ibmcep.cognitiveclass.ai/certificates/d3a570e88dc6474ba1f97f5321c5d887
- AdroIT Prolearn Certificate: IBM - Advanced Generative AI, completed on July 21, 2026. Certificate Verification ID: khfIPYurf6. Issued in association with Vellore Institute of Technology, Chennai.

APPENDIX I-B: Google Colab Notebook Links and Execution Proofs

All technical implementations, model training pipelines, and simulation codes were developed and executed in cloud-hosted Google Colab notebooks with Python 3.11 runtimes:

- Mini Project 1 Colab Notebook: Heart Disease Prediction Pipeline with ETL, 3NF SQLite database, EDA plots, Logistic Regression, Random Forest, Gradient Boosting, Stacking, SHAP analysis, and GridSearchCV hyperparameter tuning. Public Shareable Link: https://colab.research.google.com/drive/1MsfPD8QIZGPrzEL6q6jWi4zqcoR1YN_9?usp=sharing
- Mini Project 2 Colab Notebook: College Library Management System with SQLite database creation, business logic, interactive HTML frontend, 10-point test runner, latency spike simulation, and 30-day predictive maintenance metrics.

APPENDIX I-C: Core Python Source Code Extracts

Sample Implementation Code: Below are selected code extracts demonstrating key technical implementations from the project:

Listing 1: Hyperparameter Tuning Pipeline with Cross Validation

```
# Mini Project 1: Hyperparameter Tuning to Mitigate Overfitting
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import GridSearchCV, StratifiedKFold


param_grid = {
    'n_estimators': [100, 200],
    'max_depth': [None, 5, 10],
    'min_samples_split': [2, 5]
}
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
grid_search = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=param_grid, cv=cv, scoring='accuracy', n_jobs=-1
)
grid_search.fit(X_train, y_train)


best_rf = grid_search.best_estimator_
train_acc = best_rf.score(X_train, y_train)  # 0.9380
test_acc = best_rf.score(X_test, y_test)    # 0.8852
overfit_gap = train_acc - test_acc          # 0.0528 (Reduced from 0.1311)
print(f'Tuned Train: {train_acc:.4f}, Test: {test_acc:.4f}, Gap: {overfit_gap:.4f}')
```

Listing 2: ACID-Compliant Book Checkout Transaction Implementation

```
# Mini Project 2: Book Checkout Business Logic with ACID Verification
def borrow_book(conn, student_id, book_id):
    cursor = conn.cursor()
    cursor.execute(
        "SELECT available_copies FROM Books WHERE book_id = ?", (book_id,)
    )
    row = cursor.fetchone()
    if not row or row[0] <= 0:
        return False, "Error: Book is currently unavailable for borrowing."


    cursor.execute(
        "SELECT COUNT(*) FROM Borrows WHERE student_id = ? AND status = 'Active'",
        (student_id,)
    )
    if cursor.fetchone()[0] >= 3:
        return False, "Error: Maximum borrowing quota reached (3 active loans allowed)."


    due_date = (datetime.date.today() + datetime.timedelta(days=14)).isoformat()
    cursor.execute(
        "INSERT INTO Borrows (student_id, book_id, borrow_date, due_date, status) "
        "VALUES (?, ?, date('now'), ?, 'Active')",
        (student_id, book_id, due_date)
    )
    cursor.execute(
        "UPDATE Books SET available_copies = available_copies - 1 WHERE book_id = ?",
        (book_id,)
    )
    conn.commit()
    return True, f"Success: Book issued successfully. Due date: {due_date}"
```